alist, then the first occurrence is given the preference. A few examples are shown below:

```
(asoc-merge alists) ;; Syntax

(asoc-merge '((a 1) (b 2)) '((c 3) (d 4)))
((c 3) (d 4) (a 1) (b 2))

(asoc-merge '((a 1) (b 2)) '((b 3) (c 3)))
((b 3) (c 3) (a 1))

(asoc-merge '((a 1) (b 2) (a 3)) '((c 3)
(d 4)))
((c 3) (d 4) (a 1) (b 2))
```

You can use the *asoc-sort-keys* function to return a list with the elements sorted by the keys. For example:

```
(asoc-sort-keys alist) ;; Syntax

(let ((alphabets '((b . 2) (a . 1) (e .
5) (d . 4) (c . 3))))
  (asoc-sort-keys alphabets))
((a . 1) (b . 2) (c . 3) (d . 4) (e . 5))
```

Filters

The *asoc-filter* function takes a predicate function and an *alist*. It returns an *alist* with those elements that only satisfy the predicate function. In the following example, all elements whose values are less than three are returned.

```
(asoc-filter predicate alist) ;; Syntax

(let ((alphabets '((b . 2) (a . 1) (e .
5) (d . 4) (c . 3))))
  (asoc-filter (lambda (k v) (< v 3))
alphabets))
((b . 2) (a . 1))
```

The anaphoric variant of the *asoc-filter* function is *asoc–filter*. It returns *alist* elements for which the form is true. An example written using *asoc–filter* is given below:

```
(asoc--filter form alist) ;; Syntax

(asoc--filter (< value 3)
  `((b . 2) (a . 1) (e . 5) (d . 4) (c
. 3)))
((b . 2) (a . 1))
```

You can apply a predicate function to the keys using the *asoc-filter-keys* function. The *alist* elements whose keys satisfy the predicate function alone are returned. For example:

```
(asoc-filter-keys predicate alist) ;;
Syntax

(let ((alphabets `((b . 2) (a . 1) (e .
5) (d . 4) (c . 3))))
  (asoc-filter-keys (lambda (k) (eq k
'a)) alphabets))
((a . 1))
```

The *asoc-filter-values* function, on the other hand, applies the predicate function to the values of the *alist* elements. In the following example, only the *alist* elements whose values are greater than three are returned.

```
(asoc-filter-values predicate alist) ;;
Syntax

(let ((alphabets `((b . 2) (a . 1) (e .
5) (d . 4) (c . 3))))
  (asoc-filter-values (lambda (v) (> v
3)) alphabets))
((e . 5) (d . 4))
```

The *asoc-remove* function removes the elements that satisfy the predicate function and returns the rest of the *alist* elements. For example:

```
(asoc-remove predicate alist) ;; Syntax

(let ((alphabets `((b . 2) (a . 1) (e .
5) (d . 4) (c . 3))))
  (asoc-remove (lambda (k v) (< v 3))
alphabets))
((e . 5) (d . 4) (c . 3))
```

Similar to the *asoc-filter-keys* and *asoc-filter-values* functions, you have the *asoc-remove-keys* and *asoc-remove-values* functions that perform the inverse operation. Examples of the syntax and usage of these functions are given below:

```
(asoc-remove-keys predicate alist) ;;
Syntax
(asoc-remove-values predicate alist) ;;
Syntax

(let ((alphabets `((b . 2) (a . 1) (e .
5) (d . 4) (c . 3))))
  (asoc-remove-keys (lambda (k) (eq k
'b)) alphabets))
((a . 1) (e . 5) (d . 4) (c . 3))

(let ((alphabets `((b . 2) (a . 1) (e .
5) (d . 4) (c . 3))))
  (asoc-remove-values (lambda (v) (> v
3)) alphabets))
((b . 2) (a . 1) (c . 3))
```

The *asoc-partition* function can take a flattened association list that has keys and values, and return an *alist*. For example:

```
(asoc-partition flatlist) ;; Syntax
(asoc-partition `(a 1 b 2 c 3 d 4 e 5
f 6))
((a . 1) (b . 2) (c . 3) (d . 4) (e .
5) (f . 6))
```

Predicates

The *asoc-contains-key?* API accepts two arguments — an *alist* and a key — and returns a Boolean value. It returns *True* if the key is present in the *alist* and *nil* otherwise. A couple of examples are shown below:

```
(asoc-contains-key? alist key) ;;
Syntax

(asoc-contains-key? `((a 1) (b 2) (c
3)) 'a)
t
(asoc-contains-key? `((a 1) (b 2) (c
3)) 'd)
nil
```

You can use the *asoc-contains-pair?* function that ascertains if a key-value pair exists in the given *alist*. It returns